02 – Chat 1:1: frontend nad Supabase

1. Co bylo uděláno

Postavil jsem webové rozhraní chatu jeden na jednoho nad databází z kroku 01. Uživatel může:

- začít chat dvěma způsoby: tlačítkem **Message** na veřejném profilu (`/profile/:id`) nebo tlačítkem **New chat** ve stránce Chats (hledání lidí podle jména nebo username),
- přijmout nebo odmítnout **žádost o chat** (druhá strana je nejdřív `pending`),
- posílat textové zprávy, které se druhému objeví **živě** bez obnovení stránky (Supabase Realtime),
- vidět, které konverzace mají **nepřečtené** zprávy,
- filtrovat seznam chatů vyhledávacím polem v horní liště.

Vědomě vynecháno (YAGNI, „zatím jen text“): obrázky, sdílení skillu, úprava a mazání zpráv, skupiny, náhled poslední zprávy v seznamu a odznak nepřečtených v menu. Databáze je umí nebo je snadno doplní, UI zatím ne.

2. Nové a změněné soubory

Soubor	K čemu slouží
`apps/web/src/lib/chat-queries.ts`	Všechny dotazy na Supabase (selecty, insert, RPC), typy a pomocné funkce `displayName`, `formatChatTime`. Bez Reactu
`apps/web/src/hooks/useConversations.ts`	Hook: seznam konverzací a žádostí + realtime odběr
`apps/web/src/hooks/useMessages.ts`	Hook: zprávy jedné konverzace, načtení starších, odeslání, označení jako přečtené + realtime odběr
`apps/web/src/components/chat/ChatAvatar.tsx`	Kulatý avatar (nebo gradient, když fotka chybí)
`apps/web/src/components/chat/ConversationList.tsx`	Levý sloupec: tlačítko New chat, žádosti s Přijmout/Odmítnout, seznam konverzací
`apps/web/src/components/chat/ChatThread.tsx`	Pravý sloupec: hlavička, panel žádosti, seznam zpráv (bubliny), pole pro psaní
`apps/web/src/components/chat/NewChatModal.tsx`	Modální okno s hledáním lidí
`apps/web/src/pages/ChatsPage.tsx`	Stránka, která vše skládá dohromady a řeší layout desktop/mobil
`apps/web/src/App.tsx`	Route `chats/:conversationId?` (volitelný parametr)
`apps/web/src/components/layout/TopBar.tsx`	Vyhledávací pole funguje i na stránce Chats
`apps/web/src/pages/PublicProfilePage.tsx`	Tlačítko **Message** (skryté na vlastním profilu)

3. Architektura: tři vrstvy

```
Stránka (ChatsPage)          → skládá komponenty, drží URL a modal
  ├─ Komponenty (components/chat)   → jen vykreslují a hlásí kliknutí
  └─ Hooky (useConversations, useMessages) → React stav + realtime
       └─ chat-queries.ts           → čisté funkce nad supabase-js
```

Proč takhle: každá vrstva má jednu práci. `chat-queries.ts` neví nic o Reactu, takže ho jde použít odkudkoli (třeba z `PublicProfilePage` pro tlačítko Message). Komponenty nevolají databázi napřímo, takže se dají číst jako „jak to vypadá“. Stejný vzor už v projektu je (`learn-queries.ts`, `progress-queries.ts`).

4. Jak to funguje krok po kroku

A) Martin klikne na Petrově profilu na „Message“

1. `PublicProfilePage` zavolá `startDirectChat(petrId)` → RPC `get_or_create_direct_conversation`.
2. Funkce v DB vrátí id existující konverzace, nebo založí novou (Martin `accepted`, Petr `pending`).
3. Frontend přesměruje na `/dashboard/chats/{id}`.
4. `ChatsPage` načte seznam a najde v něm konverzaci, kde je Martin `accepted` → zobrazí `ChatThread` se žlutým pruhem „Waiting for Petr to accept…“. Psát se dá hned.

B) Petr dostane žádost (živě)

1. V DB přibyl řádek v `conversation_participants` pro Petra.
2. Petrův `useConversations` poslouchá změny v `conversation_participants` → přijde event → hook znovu načte seznam.
3. Konverzace je v sekci **Chat requests**. Petr zatím **nevidí žádné zprávy**: RLS `messages_select_member` vyžaduje `accepted`.
4. Klik na ✓ → `accept_conversation` → seznam se přenačte → konverzace se přesune mezi běžné a `useMessages` začne načítat zprávy.
5. Klik na ✕ → `decline_conversation` → DB celou 1:1 konverzaci smaže → u Martina přijde DELETE event a konverzace mu ze seznamu zmizí.

C) Odeslání zprávy

1. `Composer` → `send(text)` → `sendTextMessage` → `insert({ conversation_id, type: 'text', body }).select().single()`.
   - `sender_id` se nevyplňuje: má v DB výchozí hodnotu `auth.uid()` a RLS kontroluje, že sedí.
2. Vrácený řádek se hned přidá do seznamu, takže odesílatel nečeká na realtime.
3. Realtime pak stejnou zprávu pošle i odesílateli. `appendUnique` ji podle `id` zahodí, takže se nezobrazí dvakrát.
4. Příjemce má otevřený kanál `chat:{id}` s filtrem `conversation_id=eq.{id}` → zpráva se mu přidá. Protože konverzaci právě vidí, zavolá se `markRead`.
5. Trigger v DB nastavil `last_message_at`. Oba seznamy dostanou event „INSERT na messages“ a přeřadí se.
6. Když se odeslání nepovede, text v poli zůstane a pod ním se objeví chyba.

D) Nepřečtené zprávy

Konverzace je nepřečtená, když `last_message_at` (čas poslední zprávy) > `last_read_at` (kdy jsem ji naposledy otevřel). `last_read_at` posouvá RPC `mark_conversation_read`, volané při otevření konverzace a při každé příchozí zprávě, když je konverzace otevřená. Tato změna je UPDATE v `conversation_participants` → realtime → seznam se přenačte a tučné písmo zmizí.

E) Starší zprávy a scrollování

- Načte se posledních 50 zpráv (`order created_at desc limit 50` , pak `reverse()` , aby nejstarší byla nahoře).
- „Load older messages“ načte dalších 50 starších než nejstarší zobrazená (`lt('created_at', …)`). Tomu se říká **keyset (kurzorová) paginace**.
- `useLayoutEffect` v `MessageList` hlídá scroll: při přidání starších zpráv nahoru posune scroll o přírůstek výšky (obsah „neuskočí“), při nové zprávě dole sjede dolů, ale jen pokud jsem byl dole nebo jsem ji poslal já.

5. Realtime pro začátečníka

Co to je

Supabase Realtime posílá do prohlížeče přes **WebSocket** události o změnách v tabulkách (INSERT/UPDATE/DELETE). Tabulka musí být v publikaci `supabase_realtime` , což zařídila migrace 01 pro `messages` a `conversation_participants` .

```
supabase
  .channel('chat:123')                       // pojmenovaný kanál
  .on('postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'messages', filter:
'conversation_id=eq.123' },
      payload => { /* payload.new = nový řádek */ })
  .subscribe()
// při odchodu: supabase.removeChannel(channel)
```

RLS platí i pro Realtime

Server pošle event jen uživateli, který by daný řádek směl přečíst přes `SELECT` . Proto `pending` Petr nedostane event o nové zprávě, ale dostane event o svém novém řádku v `conversation_participants` . Žádná zpráva tak neunikne k někomu, kdo ji vidět nemá.

Proč „při každé změně načti vše znovu“

`useConversations` při jakémkoli eventu zavolá `fetchConversations` a nesnaží se ručně upravit stav. Je to jednodušší a vždy správné (nemůže se rozjet se stavem DB), a při desítkách konverzací je to levné. Hlídá se jen „závod“ požadavků: `requestSeq` zajistí, že starší odpověď, která dorazí později, nepřepíše novější.

Úklid

Každý `useEffect` , který otevře kanál, ho v cleanup funkci zavře (`removeChannel`). Jinak by se při přepínání konverzací kanály hromadily a zprávy by chodily vícekrát.

6. Proč dva dotazy na seznam konverzací

`conversation_participants.user_id` je cizí klíč na `auth.users`, ne na `profiles`. PostgREST umí „vnořit“ (embed) jen tabulky spojené cizím klíčem, takže `select('*, profiles(*)')` nejde. Proto:

1. `conversations` + vnořené `conversation_participants(user_id, status, last_read_at)`
2. `profiles` pro všechna id druhých stran najednou přes `.in('id', [...])`

Jsou to dva dotazy celkem, ne jeden na každou konverzaci (tím se vyhýbáme problému **N+1**).

7. Nové pojmy

Pojem	Jednoduše
Supabase Realtime	služba, která posílá změny v DB živě do prohlížeče
WebSocket	trvalé obousměrné spojení prohlížeč–server, server může posílat data sám od sebe
Channel (kanál)	pojmenovaný odběr v Realtime. Jeden kanál může poslouchat víc tabulek
`postgres_changes`	typ realtime události = změna řádku v Postgresu
Publication	seznam tabulek, jejichž změny Postgres posílá dál (Realtime je čte)
Custom hook	vlastní funkce `useNeco`, která zabalí stav + efekty, aby je komponenta jen použila
`useEffect` **cleanup**	funkce vrácená z efektu, spustí se při odchodu nebo změně závislostí (zavírání kanálů, timerů)
`useLayoutEffect`	jako `useEffect`, ale proběhne před vykreslením na obrazovku, takže je vhodný pro úpravu scrollu bez probliknutí
`useCallback`	zapamatuje funkci mezi rendery, aby se zbytečně neměnily závislosti efektů
Race condition (závod)	dva asynchronní požadavky doběhnou v jiném pořadí, než byly spuštěny
Deduplikace podle id	stejnou zprávu (z insertu i z realtime) zobrazím jen jednou
Keyset paginace	„dej mi dalších 50 starších než X" místo „přeskoč 50 a dej 50" (OFFSET). Je stabilní i při nových zprávách
N+1 problém	1 dotaz na seznam + N dotazů na detail každé položky. Řeší se jedním hromadným dotazem (`.in(...)`)
PostgREST embed	vnoření související tabulky do selectu (`conversation_participants(...)`), funguje jen přes cizí klíč
Debounce	počkám chvíli (250 ms), než uživatel dopíše, a až pak hledám, místo dotazu na každé písmeno
Volitelný parametr route	`chats/:conversationId?` = jedna route pro `/chats` i `/chats/123`, takže stránka se při přepnutí chatu znovu nemontuje

8. Co si mám sám vyzkoušet a ověřit

1. Otevřít aplikaci ve **dvou prohlížečích** (nebo normální + anonymní okno) se dvěma účty A a B.
2. A: Discover → profil B → **Message**. Ověřit žlutý pruh „Waiting…" a poslat zprávu.

3. B: bez obnovení stránky se v Chats objeví **Chat requests**. Otevřít ji: zprávy nejsou vidět, jen panel Accept/Decline.
4. B: Accept → zpráva od A se objeví. Psát si oběma směry a ověřit, že vše chodí živě.
5. A: mít otevřenou jinou stránku, B pošle zprávu, A otevře Chats → konverzace je tučně s fialovou tečkou. Po otevření tečka zmizí.
6. Vyzkoušet New chat (hledání), Enter vs. Shift+Enter, počítadlo nad 3500 znaků.
7. Založit třetí žádost a dát Decline → u odesílatele konverzace zmizí.
8. V DevTools → Network → WS najít websocket na `realtime` a podívat se na zprávy, které chodí.
9. Zkusit otevřít `/dashboard/chats/` s náhodným UUID → „This chat doesn't exist".

9. Kontrolní otázky

1. Proč `pending` uživatel nedostane realtime event o nové zprávě, ale event o žádosti ano? Co přesně rozhoduje?
2. Proč se po odeslání zpráva přidá hned z odpovědi `insert().select()` a jak se zabrání tomu, aby se po realtime eventu zobrazila dvakrát?
3. Proč `useConversations` při každé změně načte celý seznam znovu místo úpravy stavu? Kdy by to přestalo stačit?
4. K čemu je `requestSeq` v `useConversations` a co by se bez něj mohlo stát?
5. Proč nejde v jednom dotazu načíst konverzace i se jmény účastníků a jak to řeší dva dotazy bez N+1?
6. Co by se stalo, kdyby `useMessages` v cleanup funkci nezavolal `removeChannel`?